Image Classification with ResNet-18 on an Imbalanced Dataset

Transfer Learning and Class Imbalance Analysis

Santiago Freile · May 2026

Overview

This project builds a 10-class image classifier using transfer learning with a pretrained ResNet-18 model on an intentionally imbalanced version of the Imagenette dataset. The central question is not just whether the model performs well overall, but whether class imbalance causes meaningful differences in performance across categories.

The dataset contains 2,000 images with a severe imbalance: the largest class has 500 images while the smallest has 20, giving an imbalance ratio of 25. Training runs on CPU using feature extraction, where all pretrained layers are frozen and only the final classification layer is updated.

Dataset

Class Distribution

The dataset is an imbalanced version of Imagenette-10, a subset of ImageNet containing 10 visually diverse categories.

Class	Total	Train	Validation	Test
Tench	500	350	75	75
English Springer	400	280	60	60
Cassette Player	300	210	45	45
Chain Saw	250	175	37	38
Church	200	140	30	30
French Horn	150	105	22	23
Garbage Truck	100	70	15	15
Gas Pump	50	35	7	8
Golf Ball	30	21	4	5
Parachute	20	14	3	3
Total	2,000	1,400	298	302

Table 1: Class distribution across all splits. Imbalance ratio: $500/20 = 25$.

The imbalance ratio of 25 means Tench has 25 times more training examples than Parachute. This raises an important question: does the model perform equally well across all classes, or does it favor the majority?

Data Splitting

The dataset uses a stratified split to preserve class proportions across all three sets: 70% training, 15% validation, and 15% test. Stratification ensures that even minority classes like Parachute and Golf Ball are represented in each split.

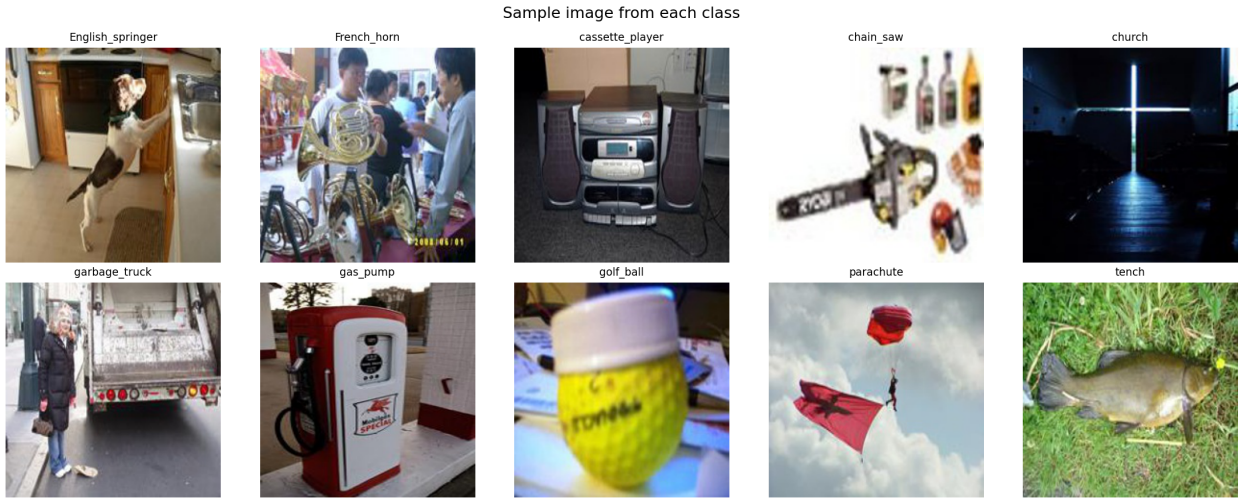

Figure 1: One sample image from each of the 10 classes.

Preprocessing

Before training, all images are resized to 224×224 to match the input expected by ResNet-18. Pixel values are normalized using ImageNet statistics:

$$x_{\text{normalized}} = \frac{x - \mu}{\sigma}, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

These values reflect the mean and standard deviation of the ImageNet training set. Using them is important because the pretrained ResNet-18 weights were learned under this normalization. Applying different statistics would shift the input distribution and degrade performance.

For the training set, a random horizontal flip is added as a simple augmentation strategy. Flipping is label-preserving, a chain saw is still a chain saw when mirrored, and helps the model generalize to new orientations without significantly increasing compute time. No augmentation is applied to the validation or test sets, keeping evaluation consistent.

After preprocessing, each batch has shape $16 \times 3 \times 224 \times 224$, corresponding to 16 images with 3 color channels at 224×224 resolution.

Model

Architecture

ResNet-18 is a convolutional neural network with 18 layers. Its defining feature is the use of residual connections, also called skip connections, which allow the output of one layer to bypass one or more intermediate layers and be added directly to a later output:

$$\text{Output} = F(x) + x$$

This design makes it easier to train deep networks by preserving gradient flow during backpropagation.

Transfer Learning Strategy

Rather than training from scratch, a pretrained ResNet-18 checkpoint is loaded. All pretrained layers are frozen so their weights are not updated during training. The final fully connected layer, which originally outputs 1,000 ImageNet class scores, is replaced with a new linear layer that outputs 10 scores for this dataset.

Property	**Value**
Architecture	ResNet-18 (pretrained on ImageNet)
Training strategy	Feature extraction (all layers frozen except final)
Trainable params	5,130
Total params	11,181,642
Input size	224×224

Table 2: Model configuration.

Freezing all but the final layer reduces the number of trainable parameters from 11 million to 5,130. This makes training feasible on CPU, completing in approximately 20 to 40 minutes for 5 epochs, depending the device.

Training Setup

Hyperparameter	**Value**
Optimizer	Adam
Learning rate	0.001
Batch size	16
Epochs	5
Loss function	CrossEntropyLoss
Device	CPU

Table 3: Training hyperparameters.

Results

Training Behavior

Epoch	Train Loss	Train Acc	Train F1	Val Acc	Val F1
1	0.9225	75.57%	0.7400	96.31%	0.9588
2	0.3138	93.21%	0.9305	98.32%	0.9829
3	0.2114	95.43%	0.9540	98.32%	0.9830
4	0.1578	96.21%	0.9620	98.99%	0.9897
5	0.1324	96.93%	0.9694	98.66%	0.9863

Table 4: Training and validation metrics per epoch.

An unusual pattern appears in the learning curves: validation accuracy is consistently higher than training accuracy across all epochs. This is the opposite of what typically happens with overfitting. The explanation is the training augmentation, random horizontal flipping makes training harder than validation, where no augmentation is applied. The model is being evaluated on cleaner inputs than it trains on.

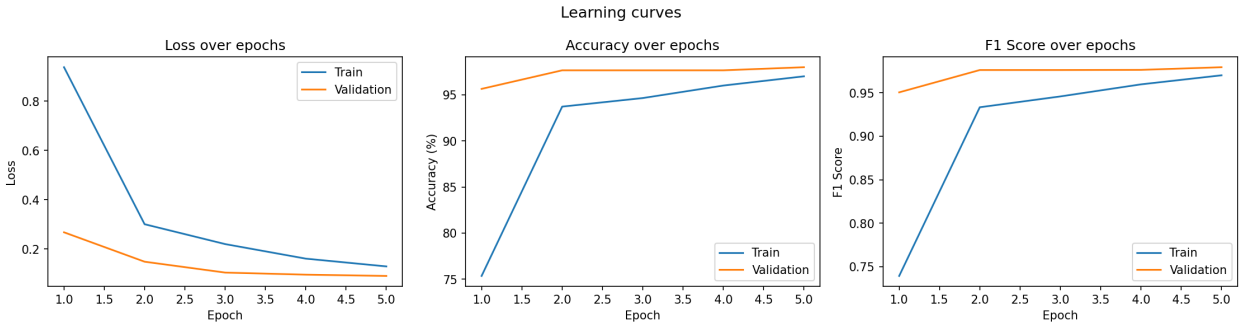

Figure 2: Learning curves across 5 epochs. Loss, accuracy, and F1 score for training and validation.

Test Set Performance

Class	Accuracy	F1 Score
English Springer	98.33%	0.9916
French Horn	91.30%	0.9130
Cassette Player	100.00%	0.9783
Chain Saw	97.37%	0.9367
Church	100.00%	1.0000
Garbage Truck	93.33%	0.9655
Gas Pump	75.00%	0.8571
Golf Ball	100.00%	1.0000
Parachute	100.00%	1.0000
Tench	98.67%	0.9933
Overall	**97.35%**	**0.9733**

Table 5: Per-class and overall results on the held-out test set.

Analysis of Class Imbalance

The results reveal that class imbalance does not hurt performance uniformly. Gas Pump, with only 35 training images and 8 test images, drops to 75% accuracy. A single wrong prediction has an outsized effect when the test set is this small.

More surprising is that Golf Ball and Parachute, both extreme minority classes with 21 and 14 training images respectively, reach 100% accuracy. The explanation is visual distinctiveness. A golf ball and a parachute look nothing like a fish, a dog, or a church. The model can classify them correctly even with very limited training data because the pretrained features are sufficient to separate them.

The main lesson is that imbalance interacts with visual similarity. A minority class that is visually distinct from everything else can still be learned well. A minority class whose features overlap with other classes will suffer.

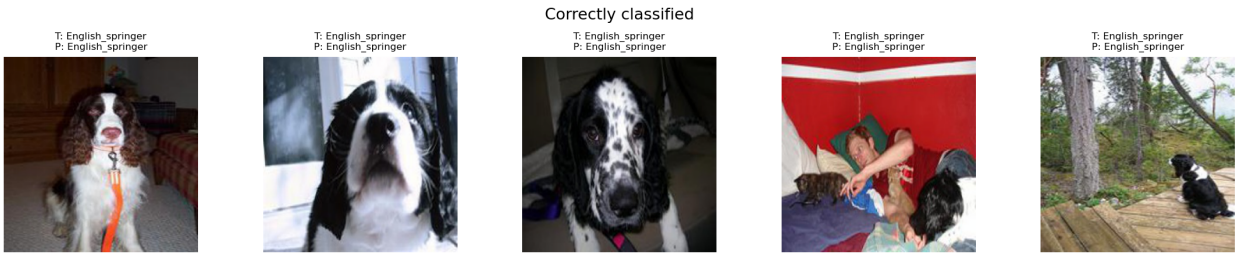

Figure 3: Five correctly classified examples from the test set.

Figure 4: Five incorrectly classified examples from the test set.

Limitations

The most important limitation is the dataset size. With only 2,000 images and classes as small as 14 training examples, evaluation on the test set is noisy. A single misclassification can shift a class accuracy by 10 to 30 percentage points. Conclusions about minority class performance should be interpreted with this in mind.

Other limitations worth noting: the model is evaluated on ImageNet-style clean images, which is a controlled setting. Performance on blurry, poorly lit, or unusually framed real-world images would likely be lower. And the training strategy, feature extraction with a frozen backbone, leaves performance on the table compared to full fine-tuning, which would be practical with a GPU.

Potential improvements include weighted loss to penalize minority class errors more heavily, oversampling to give minority classes more training examples, and full fine-tuning if compute allows.

Conclusion

This project built a complete image classification pipeline using transfer learning with ResNet-18 on an imbalanced dataset. By freezing the pretrained backbone and training only the final layer, it achieves 97.35% overall accuracy and an F1 score of 0.9733 on the test set in approximately 30 minutes on CPU.

The per-class analysis tells the more interesting story. Overall accuracy is a misleading metric on imbalanced data. The model handles minority classes well when they are visually distinctive, but struggles when limited training examples meet ambiguous visual features. Gas Pump illustrates this clearly. That gap, between a model that scores well overall and one that is actually reliable across all classes, is where the real evaluation challenge lives.